

Complexity Analysis

Big O, little o, theta, big Omega, and small omega notation

Definition

For real-valued nets f and g ,

- $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$ iff $f(x) \in O(g(x))$ and $|g(x)|$ is an asymptotic upper bound of $|f(x)|$.
- $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M |g(x)|$ iff $f(x) \in \Omega(g(x))$ and $|g(x)|$ is an asymptotic lower bound of $|f(x)|$.
- $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1 |g(x)| \leq |f(x)| \leq M_2 |g(x)|$ iff $f(x) \in \Theta(g(x))$.
- $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M |g(x)|$ iff $f(x) \in o(g(x))$.
- $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M |g(x)|$ iff $f(x) \in \omega(g(x))$.

ϵ is also denoted as $=$ for them.

Limit

If $\exists k \in \mathbb{R} : x \geq k \implies g(x) \neq 0$,

- $\limsup \left| \frac{f(x)}{g(x)} \right| < \infty$ iff $f(x) \in O(g(x))$.

Proof. $|f(x)| \leq M |g(x)|, \quad x \geq T; \left| \frac{f(x)}{g(x)} \right| \leq M, \quad x \geq T; \sup \left| \frac{f(x)}{g(x)} \right| \leq M, \quad x \geq T;$

$\limsup \left| \frac{f(x)}{g(x)} \right| \leq M < \infty;$ □

- $\liminf \left| \frac{f(x)}{g(x)} \right| > 0$ iff $f(x) \in \Omega(g(x))$.

Proof. $|f(x)| \geq M |g(x)|, \quad x \geq T; \left| \frac{f(x)}{g(x)} \right| \geq M, \quad x \geq T; \inf \left| \frac{f(x)}{g(x)} \right| \geq M, \quad x \geq T;$

$\liminf \left| \frac{f(x)}{g(x)} \right| \geq M > 0;$ □

- $\infty > \limsup \left| \frac{f(x)}{g(x)} \right| \geq \liminf \left| \frac{f(x)}{g(x)} \right| > 0$ iff $f(x) \in \Theta(g(x))$.

Proof. $|f(x)| \geq M_1 |g(x)|, \quad x \geq T; \left| \frac{f(x)}{g(x)} \right| \geq M_1, \quad x \geq T; \inf \left| \frac{f(x)}{g(x)} \right| \geq M_1, \quad x \geq T;$

$T; \liminf \left| \frac{f(x)}{g(x)} \right| \geq M_1 > 0; |f(x)| \leq M_2 |g(x)|, \quad x \geq T; \left| \frac{f(x)}{g(x)} \right| \leq M_2, \quad x \geq T;$

$\sup \left| \frac{f(x)}{g(x)} \right| \leq M_2, \quad x \geq T; \limsup \left| \frac{f(x)}{g(x)} \right| \leq M_2 < \infty;$ □

- $\lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| = 0$ iff $f(x) \in o(g(x))$. $|f(x)| < M |g(x)|; \lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| < M; \lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| = 0;$

- $\lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| = \infty$ iff $f(x) \in \omega(g(x))$. $|f(x)| > M |g(x)|; \lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| > M; \lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| = \infty;$

Relation

For any fixed $g(x)$, $o(g(x))$, $\Theta(g(x))$, and $\omega(g(x))$ are mutually exclusive.

Proof. Prove by contradiction. If $f \in o(g(x)) \cap \Theta(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M |g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R} : x \geq U \implies M_1 |g(x)| \leq |f(x)| \leq M_2 |g(x)|$; then set $M = M_1$, for $x \geq \max\{T, U\}$ $M_1 |g(x)| \leq |f(x)| < M_1 |g(x)|$; which can't happen. If $f \in \omega(g(x)) \cap \Theta(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M |g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R} : x \geq U \implies M_1 |g(x)| \leq |f(x)| \leq M_2 |g(x)|$; then set $M = M_2$, for $x \geq \max\{T, U\}$ $M_2 |g(x)| \geq |f(x)| > M_2 |g(x)|$; which can't happen. If $f \in o(g(x)) \cap \omega(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M |g(x)|$; $\forall M > 0 : \exists U \in \mathbb{R} : x \geq U \implies |f(x)| > M |g(x)|$; then set any fixed M , $M |g(x)| < |f(x)| < M |g(x)|$; which can't happen. □

For any fixed $g(x)$, $o(g(x)) \subseteq O(g(x))$;

Proof. Peak any $M > 0$. □

For any fixed $g(x)$, $\omega(g(x)) \subseteq \Omega(g(x))$;

Proof. Peak any $M > 0$. □

For any fixed $g(x)$, $\Theta(g(x)) = O(g(x)) \cap \Omega(g(x))$;

Proof. Prove $\Theta(g(x)) \subseteq O(g(x)) \cap \Omega(g(x))$; $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |f(x)| \geq N |g(x)|$; then $\exists M, N > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies N |g(x)| \leq |f(x)| \leq M |g(x)|$;

Prove $\Theta(g(x)) \supseteq O(g(x)) \cap \Omega(g(x))$; $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1 |g(x)| \leq |f(x)| \leq M_2 |g(x)|$; then $\exists M_2 > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M_2 |g(x)|$; $\exists M_1 > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M_1 |g(x)|$; □

For any fixed $g(x)$, $O(g(x)) \setminus (o(g(x)) \cup \Omega(g(x))) \neq \emptyset$;

Proof. Counterexample: $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}; g(x) = 1$;

$f \in O(g(x))$; $\exists 1 > 0, 1 \in \mathbb{R} : x \geq 1 \implies |f(x)| \leq |g(x)|$; $f \notin \omega(g(x))$; $\forall 1 > 0, T \in \mathbb{R} : \exists x \geq T, x \in \mathbb{Q} : |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$; $\forall M_1 > 0, x \in \mathbb{R}, x \notin \mathbb{Q} : M_1 |g(x)| = M_1 > |f(x)| = 0$; □

For any fixed $g(x)$, $\Omega(g(x)) \setminus (o(g(x)) \cup \Theta(g(x))) \neq \emptyset$;

Proof. Counterexample: $f(x) = 1; g(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$;

$f \in \Omega(g(x))$; $\exists 1 > 0, 1 \in \mathbb{R} : x \geq 1 \implies |f(x)| \geq |g(x)|$; $f \notin \omega(g(x))$; $\forall 1 > 0, T \in \mathbb{R} : \exists x \geq T, x \in \mathbb{Q} : |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$; $\forall M_2 > 0, x \in \mathbb{R}, x \notin \mathbb{Q} : M_2 |g(x)| = 0 < |f(x)| = 1$; □

$f(x) \in O(g(x)) \iff g(x) \in \Omega(f(x))$;

Proof. $\implies : \exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \geq \frac{1}{M} |g(x)|$; $\Leftarrow : \exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M |g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \leq \frac{1}{M} |g(x)|$; □

$f(x) \in o(g(x)) \iff g(x) \in \omega(f(x))$;

Proof. $\implies : \forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M |g(x)|$; then $\forall \frac{1}{M} > 0 : \exists T \in \mathbb{R} : x \geq T \implies |g(x)| > \frac{1}{M} |f(x)|$; $\Leftarrow : \forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M |g(x)|$; then $\forall \frac{1}{M} > 0 : \exists T \in \mathbb{R} : x \geq T \implies |g(x)| < \frac{1}{M} |f(x)|$; □

For any fixed $g(x)$, $\exists f(x) : f(x) \notin O(g(x)) \cup \Omega(g(x))$;

Proof. $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ x, & x \notin \mathbb{Q} \end{cases}; g(x) = \begin{cases} x, & x \in \mathbb{Q} \\ 1, & x \notin \mathbb{Q} \end{cases}$; For $x \in \mathbb{R} \setminus \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$; For $x \in \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M |g(x)|$; □

Binary relation

- $\Theta(g(x))$ is an equivalence class.

Proof. Transitive: $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1 |g(x)| \leq |f(x)| \leq M_2 |g(x)|$; $\exists N_1, N_2 > 0, U \in \mathbb{R} : x \geq U \implies N_1 |h(x)| \leq |g(x)| \leq N_2 |h(x)|$; then $\exists M_1 N_1, M_2 N_2 > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies M_1 N_1 |h(x)| \leq |f(x)| \leq M_2 N_2 |h(x)|$; Reflexive: $f(x) \in \Theta(f(x))$; $\exists 1, 1 > 0, a \in \mathbb{R} : x \in \mathbb{R} \implies 1 |f(x)| \leq |f(x)| \leq 1 |f(x)|$; Symmetric: $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1 |g(x)| \leq |f(x)| \leq M_2 |g(x)|$; then $\exists \frac{1}{M_2}, \frac{1}{M_1} > 0, T \in \mathbb{R} : x \geq T \implies \frac{1}{M_2} |f(x)| \leq |f(x)| \leq \frac{1}{M_1} |f(x)|$; □

- $O(g(x))$ is partial-ordered set with Θ as $=$.

Proof. Transitive: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |g(x)| \leq N |h(x)|$; then $\exists MN > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies |f(x)| \leq MN |h(x)|$; Reflexive: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \leq \frac{1}{M} |f(x)|$; Antisymmetric: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M |g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |g(x)| \leq N |f(x)|$; then $\exists M, \frac{1}{N} > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies \frac{1}{N} |g(x)| \leq |f(x)| \leq M |g(x)|$; □

- $\Omega(g(x))$ is partial-ordered set with Θ as $=$.

Proof. Similar to that of O . □

- $o(g(x))$ is a strictly partially-ordered set.

Proof. Asymmetry: $f(x) \in o(g(x)) \iff g(x) \in \omega(f(x)) \implies g \notin o(f(x))$; Transitivity: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M |g(x)|$; $\exists U \in \mathbb{R} : x \geq U \implies |g(x)| < |h(x)|$; then $\forall M > 0 : \exists \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies |f(x)| < M |h(x)|$; □

- $\omega(g(x))$ is a strictly partially-ordered set.

Proof. Similar to that of o . □

Master Theorem

Integral test (for convergence) or Maclaurin-Cauchy test

Suppose $f(x) : [a, \infty) \rightarrow \mathbb{R}$ for some $a \in \mathbb{N}$ is monotone and $\lim_{x \rightarrow \infty} f(x) = 0$. Then $\int_a^\infty f(x) dx$ is convergent iff $\sum_{k=a}^\infty f(k)$ is convergent. And,

- If $\int_a^\infty f(x) dx$ is convergent and $f(x)$ is monotone decreasing, for any $n \in \mathbb{N}_{\geq a}$, $\int_n^\infty f(x) dx \leq \sum_{k=n}^\infty f(k) \leq f(n) + \int_n^\infty f(x) dx$; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{> a}$, $f(n) + \int_n^\infty f(x) dx \leq \int_{n-1}^\infty f(x) dx$.
- If $\int_a^\infty f(x) dx$ is convergent and $f(x)$ is monotone increasing, for any $n \in \mathbb{N}_{\geq a}$, $\int_n^\infty f(x) dx \geq \sum_{k=n}^\infty f(k) \geq f(n) + \int_n^\infty f(x) dx$; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{> a}$, $f(n) + \int_n^\infty f(x) dx \geq \int_{n-1}^\infty f(x) dx$.

Proof. Consider the case where $f(x)$ is monotone increasing. The case where $f(x)$ is monotone decreasing can be proven similarly. First consider the case where $\int_a^\infty f(x) dx$ is convergent. This implies that $f(x)$ is non-positive. Thus, for any fixed $n \in \mathbb{N}_{\geq a}$, $\int_n^{n+1} f(x) dx \geq f(n)$; and for any $k \geq n + 1$, $\int_k^{k+1} f(x) dx \geq f(k) \geq \int_{k-1}^k f(x) dx$; For any $N \in \mathbb{N}_{\geq n}$, summing over all $k \in \mathbb{N}_{\geq n}$, we obtain, $\int_n^{N+1} f(x) dx \geq \sum_{k=n}^N f(k) \geq f(n) + \int_n^N f(x) dx$; Take limit $\lim_{N \rightarrow \infty}$, we

obtain $\int_n^\infty f(x) dx \geq \sum_{k=n}^\infty f(k) \geq f(n) + \int_n^\infty f(x) dx$;

Second, consider the case where $\int_a^\infty f(x) dx$ is divergent. Since $f(x)$ is monotone. Either $\int_a^\infty f(x) dx = -\infty$ or $\int_a^\infty f(x) dx = \infty$. For the former, the supremum of

$f(x)$ is non-positive, and $\sum_{k=a}^\infty f(k) = -\infty$. For the latter, there exists $x_0 > a$ such that $f(x) > 0$ for all $x \geq x_0$, and thus $\sum_{k=a}^\infty f(k) = \infty$. □

p-test (for improper integral)

For any fixed $a > 0$, $\int_a^\infty \frac{1}{x^p} dx$ converges iff $p > 1$, and if so, $\int_a^\infty \frac{1}{x^p} dx = \frac{1}{(p-1)a^{p-1}}$; $\int_a^\infty \frac{1}{x^p} dx$ converges iff $p < 1$, and if so, $\int_0^a \frac{1}{x^p} dx = -\frac{1}{(p-1)a^{p-1}}$;

Proof. $\int \frac{1}{x^p} dx = \begin{cases} -\frac{1}{(p-1)x^{p-1}}, & p \neq 1 \\ \ln(x), & p = 1 \end{cases}$

$\forall p < 1 : \nexists \lim_{x \rightarrow \infty} -\frac{1}{(p-1)x^{p-1}}; \nexists \lim_{x \rightarrow \infty} \ln(x); \forall p > 1 : \lim_{x \rightarrow \infty} -\frac{1}{(p-1)x^{p-1}} = 0 \int_a^\infty \frac{1}{x^p} dx = \frac{1}{(p-1)a^{p-1}};$

$\forall p > 1 : \nexists \lim_{x \rightarrow 0^+} -\frac{1}{(p-1)x^{p-1}}; \nexists \lim_{x \rightarrow 0^+} \ln(x); \forall p < 1 : \lim_{x \rightarrow 0^+} -\frac{1}{(p-1)x^{p-1}} = 0 \int_0^a \frac{1}{x^p} dx = -\frac{1}{(p-1)a^{p-1}};$ □

Generalized p-test (for improper integral)

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, x \in \mathbb{R}_{>0}; L_n(x) = \ln(L_{n-1}(x)), n \in \mathbb{N}, x \in \mathbb{R}_{>0}$; For any fixed $a > 1, k \in \mathbb{N}$, $\int_a^\infty \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right) (L_k(x))^m} dx$; converges iff $m > 1$.

Proof. Let $u = L_k(x); du = \frac{dx}{\prod_{i=0}^{k-1} L_i(x)}$;

$\int \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right) (L_k(x))^m} dx = \int \frac{1}{u^m} du$; By p-test, the statement holds. □

For any fixed $0 < a < 1$, $\int_0^a \frac{1}{x(\ln x)^m} dx$; converges iff $m < 1$.

Proof. By substitution of $u = \ln x$. □

Generalized p-test for series or generalized p-series test

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, x \in \mathbb{R}_{>0}; L_n(x) = \ln(L_{n-1}(x)), n \in \mathbb{N}, x \in \mathbb{R}_{>0}$; For any fixed $a \in \mathbb{N}$, $\sum_{n=a}^\infty \frac{1}{\left(\prod_{i=0}^{k-1} L_i(n)\right) (L_k(n))^m}$; converges iff $m > 1$.

Proof. By integral test and generalized p-test for improper integral, the statement holds. □

Generalized Master Theorem

For a positive valued net $T(n)$ defined on $n = b^h$ for any $h \in \mathbb{N}_0$ as $T(n) = aT\left(\frac{n}{b}\right) + f(n)$; for some constants $a > 0$ and $b > 1$, and non-negative-valued function $f(n)$,

1. If for some $c < \log_b a$, $f(n) = O(n^c)$; then $T(n) = \Theta\left(n^{\log_b a}\right)$;
2. Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, x \in \mathbb{R}_{>0}; L_n(x) = \ln(L_{n-1}(x)), n \in \mathbb{N}, x \in \mathbb{R}_{>0}$;
 - (a) If for some positive integers k and positive number m , $f(n) = \Theta\left(n^{\log_b a} \left(\prod_{i=1}^{k-1} (L_i(n))^{-1}\right) (L_k(n))^{m-1}\right)$; then $T(n) = \Theta\left(n^{\log_b a} (L_k(n))^m\right)$;
 - (b) If for some positive integer k , $f(n) = \Theta\left(n^{\log_b a} \prod_{i=1}^k (L_i(n))^{-1}\right)$; then $T(n) = \Theta\left(n^{\log_b a} L_{k-1}(n)\right)$;

(c) If for some positive integers k and some positive number m , $f(n) = \Theta\left(n^{\log_b a} \left(\prod_{i=1}^{k-1} (L_i(n))^{-1}\right) (L_k(n))^{-m-1}\right)$;
then $T(n) = \Theta\left(n^{\log_b a}\right)$;

3. If $f(n) = \Omega(n^c)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, k < 1$ such that $n \geq m$ implies $a f\left(\frac{n}{b}\right) \leq k f(n)$; is satisfied, then $T(n) = \Theta(f(n))$;

Proof. Let $s = \log_b a$, $n = b^h$, and $f(x) = x^s g(x)$. At level j , there are a^j nodes, each of size $\frac{n}{b^j}$. Thus, the total non-recursive work at level j is $a^j f\left(\frac{n}{b^j}\right)$; Hence

$$\begin{aligned} T(n) &= \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1) \\ &= \sum_{j=0}^{h-1} b^{sj} f\left(\frac{b^h}{b^j}\right) + \Theta(n^s) \\ &= n^s \sum_{j=0}^{h-1} g\left(b^{h-j}\right) + \Theta(n^s) = n^s \sum_{r=1}^h g(b^r) + \Theta(n^s) \end{aligned}$$

Case 1: If for some $c < \log_b a$, $f(n) = O(n^c)$; then $T(n) = \Theta(n^{\log_b a})$; Case 2: $L_k(b^x) = \Theta(L_{k-1}(x))$;

$$f(n) = \Theta\left(n^{\log_b a} \left(\prod_{i=1}^{k-1} (L_i(n))^{-1}\right) (L_k(n))^{m-1}\right);$$

$$g(b^r) = \Theta\left(\left(\prod_{i=1}^{k-2} (L_i(r))^{-1}\right) (L_{k-1}(r))^{m-1}\right);$$
 By

generalized p-test for series: Sub-case 1 $m > 0$: $T(n) = \Theta(n^{\log_b a} (L_k(n))^m)$; Sub-case 2 $m = 0$:

$T(n) = \Theta(n^{\log_b a} L_{k-1}(n))$; Sub-case 3 $m = 0$:

$T(n) = \Theta(n^{\log_b a})$; Case 3: If $f(n) = \Omega(n^c)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, 0 < k < 1$ such that $n \geq m$ implies $a f\left(\frac{n}{b}\right) \leq k f(n)$; is satisfied, then $a f\left(\frac{n}{b}\right) \leq k f(n)$;

$$a^j f\left(\frac{n}{b^j}\right) \leq a^{j-1} k f\left(\frac{n}{b^{j-1}}\right) \leq k^j f(n);$$

$$\begin{aligned} T(n) &= \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1) \\ &= f(n) + \sum_{j=1}^{h-1} O(k^j f(n)) + \Theta(n^s) \end{aligned}$$

$$\sum_{j=1}^{\infty} k^j f(n) = \frac{f(n)}{1-k}; T(n) = f(n) + O(f(n)) + \Theta(n^s) = \Theta(f(n)); \quad \square$$

Terms:

- h is called recursion level.
- $\log_b a$ is called critical exponent c_{crit} .
- $T(1)$ is called leaf cost.

- At each node, the first term is called recursive work, and the second term is called non-recursive work.

For original master theorem, the second case is

$$\begin{aligned} 1. \text{ If for some positive integers } m, \\ f(n) &= \Theta\left(n^{\log_b a} (\ln n)^{m-1}\right); \text{ then} \\ T(n) &= \Theta\left(n^{\log_b a} (\ln n)^m\right); \end{aligned}$$

$$\begin{aligned} 2. \text{ If } f(n) &= o\left(n^{\log_b a} (\ln n)^{-1}\right); \text{ then } T(n) = \\ &= \Theta\left(n^{\log_b a} \ln(\ln n)\right); \end{aligned}$$

$$\begin{aligned} 3. \text{ If for some positive integers } m, \\ f(n) &= \Theta\left(n^{\log_b a} (\ln n)^{-m-1}\right); \text{ then} \\ T(n) &= \Theta\left(n^{\log_b a}\right); \end{aligned}$$

Stirling's approximation

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + O\left(\frac{1}{n}\right)\right);$$

Python Operator Precedence Order

() > ** > +x -x x > * / // % > + - > « » > & ^ > | > ==
!= > > > < <= is is not in not in > not > and > or

list.py

```
6pt
a = [10, 20, 30] # list is a dynamic
                # array of elements of arbitrary types
a[0] # 10, O(1)
a[1] # 20, O(1)
a[-1] # 30, O(1)
a[-2] # 20, O(1)
len(a) # 3, O(1)
a.append(40) # a == [10, 20, 30, 40],
            # amortized O(1)
a.pop() # a == [10, 20, 30], O(1)
a.insert(1, 50) # a == [10, 50, 20, 30],
            # O(n)
a.pop(0) # a == [50, 20, 30], O(n)
a[0] = 0 # a == [0, 20, 30], O(1)
b, c, d = a # b == 0, c == 20, d == 30
a.clear() # a == []
a = [0, 1, 2]
b = a # id(a) == id(b)
b.pop() # a == b == [0, 1], id(a) == id(b)
        )
b = [1, 2] # id(a) != id(b), a == [0, 1]
a = [1, 1, 1]
a.count(1) # 3, O(n)
b = a.copy() # a == b but id(a) != id(b)
a.index(1) # 0, O(n), first occurrence
a.extend([0, 4, 3, 4]), # a == [1, 1, 1,
                        # 0, 4, 3, 4], O(k)
a.remove(4) # a.pop(a.index(x))
a.reverse() # a == [4, 3, 0, 1, 1, 1], O(
                n)
```

```
a.sort() # [0, 1, 1, 1, 3, 4], O(n log n)
        # worst-case O(n) best-case, Powersort
sorted(a) # same as a.sort()
del a # a is now not defined
a = [1, "1", [1]] # arbitrary types
```

array.py

```
6pt
from array import array
a = array('b', [10, 20, 30]) # array.array
    # is a dynamic array of elements of
    # the same primitive data type in a
    # compact, contiguous memory buffer
# 'b' char
# 'B' unsigned char
# 'h' short
# 'H' unsigned short
# 'i' int
# 'I' unsigned int
# 'l' long
# 'f' float
# 'd' double
a.itemsize # 1
a.buffer_info() # (address, len(a))
b = a.tolist() # b == [10, 20, 30]
# As len(a) gets large, array tends to be
# smaller than list
# Methods for list are available for array
# with same complexity
```

linear_data_structures.py

```
6pt
class SNode:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next

class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def __len__(self):
        return self.size

    def is_empty(self):
        return self.head is None

    def prepend(self, x):
        self.head = SNode(x, self.head)
        self.size += 1

    def append(self, x):
        node = SNode(x)
        if self.head is None:
            self.head = node
        else:
            cur = self.head
            while cur.next is not None:
                cur = cur.next
            cur.next = node
        self.size += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("pop from
                empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
        else:
            self.head.prev = None
        return x

    def pop_back(self):
```

```
if self.head is None:
    raise IndexError("remove from
        empty list")
x = self.head.data
self.head = self.head.next
self.size -= 1
return x

def __iter__(self):
    cur = self.head
    while cur is not None:
        yield cur.data
        cur = cur.next
```

class DNode:

```
def __init__(self, data):
    self.data = data
    self.prev = None
    self.next = None
```

class DoublyLinkedList:

```
def __init__(self):
    self.head = None
    self.tail = None
    self.size = 0
```

```
def __len__(self):
    return self.size
```

```
def is_empty(self):
    return self.size == 0
```

```
def prepend(self, x):
    node = DNode(x)
    node.next = self.head
    if self.head is None:
        self.tail = node
    else:
        self.head.prev = node
    self.head = node
    self.size += 1
    return node
```

```
def append(self, x):
    node = DNode(x)
    node.prev = self.tail
    if self.tail is None:
        self.head = node
    else:
        self.tail.next = node
    self.tail = node
    self.size += 1
    return node
```

```
def pop_front(self):
    if self.head is None:
        raise IndexError("pop from
            empty list")
    x = self.head.data
    self.head = self.head.next
    self.size -= 1
    if self.head is None:
        self.tail = None
    else:
        self.head.prev = None
    return x
```

```
def pop_back(self):
```

```
if self.tail is None:
    raise IndexError("pop from
        empty list")
x = self.tail.data
self.tail = self.tail.prev
self.size -= 1
if self.tail is None:
    self.head = None
else:
    self.tail.next = None
return x
```

```
def __iter__(self):
    cur = self.head
    while cur is not None:
        yield cur.data
        cur = cur.next
```

class ListStack:

```
def __init__(self):
    self.data = []

def __len__(self):
    return self.data.__len__()
```

```
def push(self, x):
    self.data.append(x)
```

```
def pop(self):
    if self.is_empty():
        raise IndexError("pop from
            empty stack")
    return self.data.pop()
```

```
def peek(self):
    if self.is_empty():
        raise IndexError("peek at
            empty stack")
    return self.data[-1]
```

```
def is_empty(self):
    return self.data.__len__() == 0
```

class LinkedStack:

```
def __init__(self):
    self.data = SinglyLinkedList()
```

```
def __len__(self):
    return self.data.__len__()
```

```
def push(self, x):
    self.data.prepend(x)
```

```
def pop(self):
    if self.is_empty():
        raise IndexError("pop from
            empty stack")
    return self.data.pop_front()
```

```
def peek(self):
    if self.is_empty():
        raise IndexError("peek at
            empty stack")
    return self.data[0]
```

```
def is_empty(self):
    return self.data.__len__() == 0
```